msieve GPU fork: build options and runtime knobs

Stefan Teleman <stefan.teleman@gmail.com>
in collaboration with Claude Opus 5.5 (Anthropic)

November 09 2026

Abstract

This is a reference for people who build the GPU fork of msieve from source. It lists every Makefile variable, every compile-time macro and every environment variable that the fork adds or relies on, with its values, its default, its effect, and the source line that reads it. The command-line options whose meaning the fork changed are listed at the end, followed by an alphabetical index.

Contents

1 Introduction **2**
1.1 Audience and scope . . . 2
1.2 How to read an entry . . . 2
1.3 Unset means default; how values are parsed . . . 3
1.4 Statements about hardware . . . 3

2 Building **3**
2.1 Requirements . . . 3
2.2 Makefile variables . . . 4
2.3 Example make lines . . . 6
2.4 PTX files and the current-directory rule . . . 6

3 Compile-time macros **6**
3.1 Macros that can be overridden . . . 6
3.2 Macros set by the Makefile . . . 8
3.3 Macros set by the source . . . 8
3.4 Interface numbers . . . 8
3.5 Fixed tuning constants . . . 8

4 Environment variables **10**
4.1 Polynomial selection, stage 1 . . . 10
4.2 Polynomial selection, size optimization . . . 10
4.3 Polynomial selection, root optimization . . . 11
4.4 Line siever . . . 13

4.5 Lattice siever . . . 14
4.6 Batch factoring . . . 15
4.7 Filtering and the relation store . . . 15
4.8 Linear algebra . . . 16
4.9 Square root . . . 17
4.10 GPU selection, several GPUs, MPI . . . 18
4.11 The NFS by default . . . 18
4.12 Testing and debugging: summary . . . 19

5 Command-line options **20**

6 README and code disagreements **20**

7 Alphabetical index **21**

1 Introduction

1.1 Audience and scope

The fork is msieve trunk r1113 (git commit `6d4fa7e`) with about ninety commits on top. It adds GPU polynomial selection (stage 1, size optimization, root optimization), a GPU line siever and a GPU lattice siever, GPU block Lanczos with MPI and optional NCCL, parallel filtering with an in-memory relation store, a square root that runs several dependencies at once and does its large products on the GPU, and the number field sieve by default in GPU builds. `README.md` summarizes the fork and `msieve-gpu-design.pdf` explains it. This document only lists the settings.

Every default and every effect below was read from the code. A source location `file:line` is the line that reads the setting, at commit `39e5d32` (HEAD when this was written; it changed only the design document, so the source files and line numbers are those of `42d95c1`). The upstream options of msieve (the `-np`, `-ns`, `-nc` option strings, filtering bounds, MPI grid) are described in `Readme.nfs` and are not repeated here.

1.2 How to read an entry

Each entry has these fields:

Builds the builds in which the setting has an effect: all, `CUDA=12/13` (a GPU build), `MPI=1`, `NCCL=1`, or non-Windows.
Values what the setting accepts and how the code parses it.
Default the value used when the setting is absent.
Effect what changes.
Use when to change it.
Source where the code reads it.

Tags: [test/debug] marks a setting for testing, debugging or comparison, not for production runs; [slow] marks one that makes a run noticeably slower; [measurement] marks one that only writes measurement output.

1.3 Unset means default; how values are parsed

An environment variable that is not set always gives the default. A variable that is set, even to an empty string, is read by the rule of its parser, and the rules differ from variable to variable. Table 1 lists the five rules. The difference matters: for example `MSIEVE_CPU_LIFT=0` turns CPU lifting *on*, because only the presence of the variable is tested, and `MSIEVE_RELSTORE=no` turns the relation store *off*, because `atoi("no")` is 0.

Rule	Meaning
presence	set to anything (also `0` or empty): on. Unset: off.
`strcmp "0"`	exactly the string `0`: off. Anything else, including `no`, `off` and empty: on.
`strcmp "1"`	exactly the string `1`: on. Anything else: off.
`atoi` test	the value as an integer (`atoi`; a non-number and the empty string are 0): 0 means off, anything else on.
number	`atoi`, `atol`, `strtol`, `strtoul`, `strtoull` or `atof`; a non-number reads as 0, and most entries then clamp the value (stated per entry).
file name	the value is a path; the file is opened for writing (`"w"`) or appending (`"a"`).

Table 1: How the fork parses environment variables.

Most settings are read once, the first time the code needs them, so they must be set before msieve starts. With MPI, pass them to every process (`mpirun -x NAME`).

1.4 Statements about hardware

All measurements behind the defaults were made on one machine: an NVIDIA RTX PRO 4500 Blackwell (sm_120, 32 GB) and an Intel i9-13900K (32 threads). The production targets are the RTX PRO 6000 Blackwell (the minimum configuration) and the B200 (the optimal one). Some defaults were chosen for those targets although they are slower on the development card; this is said where it applies. **Every statement about the RTX PRO 6000 or the B200 in this document is an untested estimate.** The builds include sm_90 and sm_100 code (Hopper, B200), but it has not run on such hardware.

2 Building

2.1 Requirements

GMP	always (`-lgmp`). The GPU square root needs 64-bit GMP limbs; with 32-bit limbs it uses GMP only (`gnfs/sqrt/sqrt_gpu_mul.c:180`).
zlib	unless `NO_ZLIB=1`.
pthreads, libdl	always on Linux (`-lpthread`, `-ldl`).
CUDA toolkit	for `CUDA=12` or `CUDA=13`: `nvcc` must be on `PATH`; the build links the driver stub `libcuda.so`. The program uses the driver API only (no runtime library).
CGBN	headers only, for `CUDA=`: the fork at `github.com/steleman/nvidia-cgbn` (it adds signed arithmetic). Location `CGBN_INCLUDE`, default `/usr/local/include`.
GMP-ECM	only with `ECM=1` (`-lecm`).
MPI	only with `MPI=1`: an `mpicc` on `PATH`. A CUDA-aware MPI (for example HPC-X) is optional; it lets the GPU linear algebra pass device buffers.
NCCL	only with `NCCL=1` (needs `CUDA=` and `MPI=1`); `NCCL_HOME` if it is not in the system paths.

Plain `make` prints help; `make all` builds `libmsieve.a` and `./msieve`.

2.2 Makefile variables

CUDA

Values empty (CPU build), 12 or 13.

Default empty: a CPU-only build.

Effect Any non-empty value adds -DHAVE_CUDA, the CUDA include directory and the driver stub, compiles the GPU sources, and builds the PTX files and cub/ (Makefile:54-81). The value selects the PTX architectures (table 2). CUDA=13 builds every PTX family for sm_80, 86, 89, 90, 100 and 120. CUDA=12 builds them for sm_60, 61, 70 and 75 and, since this fork's last fix, also for sm_80, 86, 89, 90, 100 and 120 (CUDA 12.8 or later is needed for sm_100/120); before, it built sm_60–75 only, and on a compute 8.0 or newer card stage 1 stopped with a CUDA error because the loaders ask for *_sm80.ptx ... *_sm120.ptx.

Use CUDA=13 for Ampere and newer (the tested configuration, CUDA 13.2). CUDA=12 with CUDA 12.9 was built and ran a c85 from scratch on the RTX PRO 4500 (all GPU stages, factors correct). Compute 5.x is asked for *_sm50/52.ptx, which no build writes.

Source Makefile:54-154.

Build	PTX architectures	cub/ (GPU_SM)
CUDA=12	sm_60, 61, 70, 75, 80, 86, 89, 90, 100, 120	600, ..., 890, 900, 1000, 1200
CUDA=13	sm_80, 86, 89, 90, 100, 120	800, 860, 890, 900, 1000, 1200

Table 2: Architectures per CUDA build (Makefile:83-152).

NVCC, CUDA_ROOT, CUDA_LIBS, CUDA_PATH

Builds CUDA=.

Values paths.

Default Linux: NVCC = $(shell which nvcc), CUDA_ROOT its directory's parent, CUDA_LIBS = $(CUDA_ROOT)/lib64/stubs/libcuda.so. Windows (WIN=1): CUDA_ROOT from the environment variable CUDA_PATH, cuda.lib under lib/x64 (WIN64=1) or lib/win32.

Effect The toolkit is the first nvcc on PATH; it decides the CUDA version macros the C code sees (CUDA_VERSION >= 13000 selects the CUDA 13 cuCtxCreate signature).

Use Put the wanted toolkit first on PATH, e.g. PATH=/usr/local/cuda-13.2/bin:$PATH make all CUDA=13. make NVCC=... also works.

Source Makefile:59-73.

CGBN_INCLUDE

Builds CUDA=.

Values a directory that contains cgbn/cgbn.h.

Default /usr/local/include (?=, so the environment can set it too).

Effect Include path for sizeopt_core.cu (size and root optimization kernels) and lasieve_core.cu (lattice siever's la_cofactor).

Use When CGBN is installed elsewhere.

Source Makefile:509-513, 530.

GPU_LATTICE

Builds CUDA=.

Values 1 or anything else.

Default 1 (?=).

Effect 1 adds -DRS_GPU_LATTICE: the root sieve's lattice choice (kernels lt_xy, lt_x) runs on the GPU on the roots kept there, for degrees 4 and 5, when the GPU root sieve setup and the GPU line sieve are both on (root_sieve_gpu.c:2344-2359). Any other value: the lattice choice runs on the CPU. Results are bit-identical.

Use Keep 1: it is the default for the target hardware. On the RTX PRO 4500 it was about 4% slower than GPU_LATTICE=0; that it is faster on the RTX PRO 6000 or B200 is an untested estimate.

Source Makefile:46, 76-80.

ECM

Values 1 or not.
Default off.
Effect -DHAVE_GMP_ECM, links -lecm: msieve's ECM for small factors (upstream).
Source Makefile:156-159.

MPI

Values 1 or not.
Default off.
Effect CC = mpicc and -DHAVE_MPI. With MPI the fork splits polynomial selection over the processes, divides each node's GPUs and CPUs among its processes (MPI_Comm_split_type(MPI_COMM_TYPE_SHARED), common/driver.c:252-266), and runs the GPU block Lanczos on an MPI grid. All MPI calls are on the main thread.
Use Several nodes or several processes per node. Run with mpirun --bind-to none, or OpenMPI pins all threads of a process to one core. With HPC-X, build with OMPI_CC=gcc make all CUDA=13 MPI=1 (its mpicc would use nvc).
Source Makefile:165-168.

NCCL, NCCL_HOME

Builds meaningful with CUDA= and MPI=1.
Values NCCL=1; NCCL_HOME a directory with include/ and lib/.
Default off; NCCL_HOME empty (system paths).
Effect -DHAVE_NCCL, links -lnccl (with -L and an rpath to $(NCCL_HOME)/lib if set). It only makes NCCL available; it is used at run time with MSIEVE_LA_NCCL=1.
Use Multi-GPU linear algebra with one GPU per process. The NCCL transport is untested: NCCL refuses two processes on one GPU, and the development machine has one GPU.
Source Makefile:169-179.

VBITS

Values 64, 128, 256 (anything else is a compile error, common/lanczos/lanczos.h:34).
Default 64.
Effect Block Lanczos vector width, passed as -DVBITS to the C code and to lanczos_core.cu (Makefile:536).
Use 128 and 256 are correct on the GPU but were not faster on the RTX PRO 4500 (600k matrix: 15.8 / 16.3 / 17.1 s). The lanczos_core_sm*.ptx file name does not include VBITS and the loader has no interface check, so never mix PTX files from builds with different VBITS.
Source Makefile:18, 34, 536.

NO_ZLIB, BOINC, WIN, WIN64

Effect Upstream options. NO_ZLIB=1: -DNO_ZLIB, no -lz. BOINC=1: -DHAVE_BOINC and the BOINC libraries (BOINC_INC_DIR, BOINC_LIB_DIR, both .). WIN=1/WIN64=1: Windows (MinGW/MSYS2) paths, no -ldl.
Use The fork was built and tested on Linux only. The Visual Studio project directories (build.vc*, build.cuda.vc*, build.vs) do not list the fork's new source files. On Windows the parallel filtering, relation-store parsing, batch-factoring threads and parallel square root are compiled out (they need pthread/sysconf).
Source Makefile:16-17, 59-68, 160-164, 180-191.

MACHINE_FLAGS

Values extra compiler flags.
Default empty.
Effect Appended to CFLAGS for every C file and the link of msieve. It is *not* passed to nvcc.
Use The way to set the overridable macros of section 3, e.g. make all CUDA=13 MACHINE_FLAGS=-DQ_ROOTS_MIN=1.
Source Makefile:48.

Other fixed flags: STD_FLAGS = -std=c11 -D_GNU_SOURCE -D_DEFAULT_SOURCE (-std=c11 also turns off GCC's floating-point contraction; size optimization results depend on it), OPT_FLAGS = -O3 -march=native -DNDEBUG ..., NVCC_FLAGS = -O3 -lineinfo -ptx, and -fmad=false

for `sizeopt_core.cu` (so the GPU rounds like the CPU code) (`Makefile:30-38, 56, 510`). The GPU size and root optimization reproduce the CPU results bit for bit only against a build without fused multiply-adds. `MSIEVE_SVN_VERSION` comes from `svnversion` (`Makefile:41-44`).

2.3 Example make lines

```
export PATH=/usr/local/cuda-13.2/bin:$PATH
make all CUDA=13                          # GPU build (tested configuration)
make all CUDA=13 ECM=1                    # with GMP-ECM
make all CUDA=13 GPU_LATTICE=0            # root sieve lattice choice on the CPU
make all CUDA=13 CGBN_INCLUDE=$HOME/cgbn/include
make all CUDA=13 MPI=1                    # MPI (Fedora OpenMPI: not CUDA-aware)
OMPI_CC=gcc make all CUDA=13 MPI=1        # HPC-X, after hpcx_load
make all CUDA=13 MPI=1 NCCL=1 NCCL_HOME=/opt/nccl
make all CUDA=13 VBITS=128
make all CUDA=13 MACHINE_FLAGS="-DQ_ROOTS_MIN=1"
make all                                  # CPU only
```

Run `make clean` whenever a variable changes. The Makefile does not track flags: objects built with other flags stay. `make clean` removes the GPU-only objects in every configuration (`Makefile:432-436`); a stale non-MPI `lanczos_gpu.o` once broke the MPI build.

2.4 PTX files and the current-directory rule

A GPU build writes six families of PTX files into the top directory (table 3). **msieve loads them by file name from the current working directory at run time** (`cuModuleLoad("stage1_core_sm120.ptx")` and so on), not from the directory of the binary. Copy the `.ptx` files of the same build into every run directory. A PTX file from an older build is loaded without complaint by four of the six loaders and runs old kernels, or fails. The driver compiles PTX on first use (about 7 s for the size-optimization file) and caches the result.

Each loader maps the device's compute capability to a file: 12.x → sm_120, 10.x → sm_100, 9.x → sm_90, 8.0–8.5 → sm_80, 8.6–8.8 → sm_86, 8.9 → sm_89, 7.0–7.4 → sm_70, 7.5 → sm_75, 6.0 → sm_60, 6.1+ → sm_61.

3 Compile-time macros

3.1 Macros that can be overridden

These are guarded by `#ifndef` (or required) and can be set with `MACHINE_FLAGS`. `MACHINE_FLAGS` reaches the C compiler only; none of these is read by a `.cu` file except `VBITS`, which the Makefile passes to `nvcc` itself.

`MIN_NFS_BITS`

Values	an integer number of bits.
Default	264 (msieve's; about 79.5 digits).
Effect	Inputs of at most this many bits are never factored with the NFS (`common/driver.c:97`, also msieve's own size checks).
Use	[test/debug] Measurements below 80 digits only (the crossover study used `-DMIN_NFS_BITS=230`). msieve has no polynomial parameters below about 75 digits and the NFS failed at 75–76 digits; the gain at 78–79 digits was 17–27 s.
Source	`include/common.h:264`.

PTX family	Used by; check; what happens if it cannot be used	Loader
`stage1_core`	stage 1 collision search. No check. Missing: CUDA error, exit.	`stage1_sieve_gpu.c:1700`
`sizeopt_core`	GPU size optimization and the whole GPU root optimization. No check. Missing: log line, CPU code.	`stage1_sizeopt_gpu.c:89`, `root_sieve_gpu.c:2253`
`sieve_core`	GPU line siever. No check. Missing: CPU line siever.	`sieve_line_gpu.c:63`
`lasieve_core`	GPU lattice siever. Checks `la_abi` = {`LA_ABI`, `sizeof(la_batch_t)`}; mismatch or missing: log line, then the line siever (only when the lattice siever was the default, see `-ns`).	`sieve_lattice_gpu.c:1028-1066`
`lanczos_core`	GPU block Lanczos. No check (not even `VBITS`). Missing: CPU Lanczos.	`lanczos_gpu.c:66, 164`
`sqrtmul_core`	GPU products in the square root. Checks `sm_abi` = {`SM_ABI`, `sizeof(sm_crt_t)`}; mismatch or missing: GMP only.	`sqrt_gpu_mul.c:145, 210-226`

Table 3: PTX files (`<family>_sm<arch>.ptx`).

`Q_ROOTS_MIN`

Builds `CUDA=`.
Values an integer expression (the macro is used where `degree` is in scope).
Default `degree`: a special-q must have as many roots as the polynomial degree.
Effect Fewest roots of a special-q in the GPU stage 1 search. `1` searches every special-q (the CPU code's choice): more hits (c120 test coefficient: 2,588,149 distinct hits instead of 1,580,311) and more stage 1 time. The default won the end-to-end test.
Use `MACHINE_FLAGS=-DQ_ROOTS_MIN=1` for a full search.
Source `gnfs/poly/stage1/stage1_sieve_gpu.c:28-30, 2222, 2300`.

`LA_TILE_POS`

Builds `CUDA=`.
Values positions per `la_tile` tile; at most `LA_TILE_POS_MAX` = 16384; the code uses the larger of `LA_TILE_POS` and I and needs whole rows.
Default 8192 (three `la_tile` blocks per SM; it was 16384).
Effect Shared-memory tile of the lattice siever's small-prime kernel. Thresholds are set per 16-position run, so relations differ slightly between values (they all validate). Only the host reads it (`sieve_lattice_gpu.c:594`); the kernel takes it from the batch, so `MACHINE_FLAGS` suffices.
Use Tuning on other GPUs.
Source `gnfs/sieve/lasieve_core.h:53-56`.

`VBITS` (macro)

Effect Required by `lanczos.h:28` and `lanczos_core.cu:35`. Set it with the make variable `VBITS`, never with `MACHINE_FLAGS` (the Makefile already passes `-DVBITS=$(VBITS)`).

Macro	Set by	Effect
HAVE_CUDA	CUDA=	all GPU code paths; NFS by default; GPU poly time scale
RS_GPU_LATTICE	GPU_LATTICE=1 (default)	root sieve lattice choice on the GPU
HAVE_MPI	MPI=1	MPI paths
HAVE_NCCL	NCCL=1	NCCL transport for the GPU Lanczos rings
HAVE_GMP_ECM	ECM=1	GMP-ECM (upstream)
NO_ZLIB	NO_ZLIB=1	no gzip savefiles (upstream)
HAVE_BOINC	BOINC=1	BOINC wrapper (upstream)
MSIEVE_SVN_VERSION	always	version string from svnversion

Table 4: Macros from the Makefile (Makefile:48-191).

3.2 Macros set by the Makefile

3.3 Macros set by the source

On every platform except Windows the source defines BATCH_THREADS (common/batch_factor.c:20), SINGLETON_THREADS (common/filter/singleton.c:413), RELPAR_THREADS (gnfs/filter/relpar.c:11), RELSTORE_TLS (gnfs/relstore.c:11) and SQRT_THREADS (gnfs/sqrt/sqrt.c:261). They switch on the threaded batch factoring, singleton removal, relation parsing, relation-store parsing and square root; without them the related environment variables have no effect. They are not meant to be set by hand. GPU_DEBUG (stage1_sieve_gpu.c:22, commented out, as upstream) [test/debug] turns on stage 1 debug printing.

3.4 Interface numbers

LA_ABI (3, gnfs/sieve/lasieve_core.h:48) and SM_ABI (1, gnfs/sqrt/sqrtmul_core.h:37) are compiled into the PTX and into the binary and compared at load time (table 3). They are not user settings: a developer raises them when a kernel's arguments change.

3.5 Fixed tuning constants

These are plain #defines (no #ifndef); changing them means editing the source and, for the ones shared with a .cu file, rebuilding the PTX.

Constant (file)	Value	Meaning
NFS_DEFAULT_DIGITS (common/driver.c:84)	80	default of MSIEVE_NFS_MIN_DIGITS
MAX_GPU (include/cuda_xface.h:27)	64	devices recorded and lanes
FOUND_ARRAY_SIZE (stage1_core.h:25)	2^{20}	stage 1 collisions per batch (was 1000; the rest were dropped)
LIFT_MAX_LIMBS (stage1_sieve_gpu.c:24)	64	GPU special-q lifting limit (32-bit limbs of trans_N, trans_m0)
GPU memory share (stage1_sieve_gpu.c:1895)	30%	stage 1 budget per GPU, divided by its sharers and threads; gpu_mem_mb= in -np overrides
SO_THREADS, SO_ROUNDS (stage1_sizeopt_gpu.c:40-41)	128, 10	size optimization block size, minimize rounds

Constant (file)	Value	Meaning
flush target (stage1_sizeopt_gpu.c:299)	8×	collected candidates per flush, times the resident thread count
RO_THREADS_PER_BLOCK (root_sieve_gpu.c:80)	128	root optimization so_basic block
RS_MAX_ROOTS, RS_MAX_POWERS (alpha_core.h:149-150)	8192, 64	root sieve setup on the GPU
RF_MAX_JOBS (alpha_core.h:115)	64	final rotations per request
LS_MAX_TABLE (line_sieve_core.h:50)	25600	root optimization line sieve tables; with LS_CAP 1536 it sets the shared memory the ls_sieve kernel needs
LS_SEG, LS_CHUNK (gnfs/sieve/sieve_core.h:15-16)	16384, 2^{20}	GPU line siever segment and bucket chunk
LS_LINES, LS_MAX_CANDS (sieve_line_gpu.c:33-34)	16, 2^{21}	GPU line siever lines per batch, candidates per chunk (an overflowing chunk is redone line by line)
LA_CANDS (sieve_lattice_gpu.c:40)	49152	lattice siever candidates per lattice
batch size (sieve_lattice_gpu.c:1113)	≤ 64	lattices per batch: $\min(64, 0.4 \cdot$ free memory/per lattice); MSIEVE_LATTICE_BATCH overrides
LA_RESIEVE_MIN (lasieve_core.h:27)	256	primes resieved on the GPU; also the cap of MSIEVE_LATTICE_TINY
LA_TILE_POS_MAX (lasieve_core.h:53)	16384	upper bound of LA_TILE_POS
LA_COF_BITS, LA_COF_TPI (lasieve_core.h:30-31)	256, 4	la_cofactor CGBN width and threads per candidate
RELPAR_BLOCK (gnfs/filter/relpar.h:21)	16384	relation lines per parallel parse block
PREP_CHUNK (gnfs/relstore.c:333)	262144	relation-store parse chunk
SM_NP, SM_LOG2_MAX (sqrtmul_core.h:31-32)	3, 40	square root NTT: three primes $c \cdot 2^{40} + 1$

4 Environment variables

4.1 Polynomial selection, stage 1

`MSIEVE_POLY_TIME_SCALE`

Builds `CUDA=`.
Values number (`atof`).
Default 0.0078125 (1/128).
Effect Multiplies msieve's polynomial selection time limit (`params.deadline`), rounded, at least 1 s. A value ≤ 0 or a non-number leaves msieve's limit unchanged (the same as `1`). A `poly_deadline=` in the `-np` string is applied afterwards and overrides it. The limit is also a wall-clock end: past it stage 1 stops and survivors are no longer root-optimized.
Use `1` for msieve's full CPU-era limits. 1/128 found the same polynomials as 1/32 at c100–c120; the earlier defaults were 1/4, then 1/32.
Source `gnfs/poly/poly_param.c:205-211`.

`MSIEVE_Q_THREADS`

Builds `CUDA=`.
Values number (`strtol`), clamped to 0–256.
Default 4.
Effect Threads that enumerate special-q into a ring of batches the GPU drains. `0`: the original serial loop in the stage 1 thread.
Use More producers measured no faster.
Source `gnfs/poly/stage1/stage1_sieve_gpu.c:2033-2037, 2225`.

`MSIEVE_CPU_LIFT` [test/debug]

Builds `CUDA=`.
Values presence.
Default unset: special-q roots are lifted mod q^2 on the GPU when the fast path applies.
Effect Set to anything (also `0`): lift on the CPU.
Use Comparison with the old code path only.
Source `stage1_sieve_gpu.c:2193`.

`MSIEVE_OLD_CRT` [test/debug]

Builds all (`stage1_roots.c` is shared).
Values presence.
Default unset: incremental Garner CRT for $p < 2^{32}$ with several factors.
Effect Set to anything (also `0`): msieve's original root combination.
Use Comparison only.
Source `gnfs/poly/stage1/stage1_roots.c:853-856`.

4.2 Polynomial selection, size optimization

`MSIEVE_GPU_SIZEOPT`

Builds `CUDA=`; degrees 4 and 5; only when stage 1 runs in the same process (`-np` or `-np1`).
Values `strcmp "0"`.
Default on.
Effect On: collisions are rebuilt, size-optimized and scored on the GPU; about 1 in 4000 returns to the host. `0`: size optimization on the CPU (stage 2 threads). Survivors are the same (bit-identical with the no-FMA CPU build).
Use `0` to compare, or if `sizeopt_core_sm*.ptx` cannot be used (that also falls back by itself).
Source `gnfs/poly/poly_skew.c:354-360, 879-885`.

MSIEVE_SIZEOPT_LOG [measurement]

Builds all.
Values file name, opened for appending. MPI ranks $r > 0$ append to `<name>.mpiNN`.
Default unset: no log.
Effect Writes `S` and `P` lines per size-optimization survivor (time, hits, norm, alpha; the polynomial as a `.ms` line), `H` every 100,000 stage 1 hits, and `R c_d Y1 E size alpha` per root optimization.
Use Comparing size optimization between builds (`grep "^P "`); measuring how well the size score predicts E.
Source `gnfs/poly/poly_skew.c:188-205`.

MSIEVE_STAGE2_THREADS

Builds all.
Values number (`strtol`), clamped to 1–256.
Default online CPUs divided by the MPI processes on the node.
Effect Workers for size and root optimization, each with its own state; also used by the `-nps`/`-npr` file paths.
Use Lower it to leave CPUs to other work.
Source `gnfs/poly/poly_skew.c:415-433`.

4.3 Polynomial selection, root optimization

MSIEVE_GPU_ROOTOPT

Builds `CUDA=`; degrees 4–6.
Values `strcmp "0"`.
Default on.
Effect On: root optimization in the GPU engine (batched root sieve in coroutines, kernels for `optimize_basic`, line sieve, setup, lattice choice, alpha, size score, Murphy E). `0`: msieve's CPU code (same best E; 97–100% of the lines identical).
Source `gnfs/poly/poly_skew.c:62-72`.

MSIEVE_ROOTOPT_TOPK

Builds all (different defaults).
Values number (`strtoul`); `0` means all.
Default 48 in `CUDA=` builds, 0 otherwise.
Effect A size-optimized polynomial is root-optimized only if its size score (norm$\cdot e^{\alpha}$, lower is better) is among the K best seen so far in the run. It applies to polynomials passed from size to root optimization in one run; `-npr` on a `.ms` file root-optimizes every line.
Use It bounds the work rather than improving polynomials (the size score hardly predicts the final E: Spearman -0.09 at c120). `0` with long time limits.
Source `gnfs/poly/poly_skew.c:241-281, 315`.

MSIEVE_POLY_KEEP_E

Builds all.
Values number (`atof`); used only if $0 < x < 1$, otherwise 1.
Default 0.25.
Effect Polynomials are kept down to x times msieve's minimum E, so the best one is used even when none reaches the minimum (instead of a second search). Degree 5's Bernstein bound scales with it too. If nothing at all is kept, `find_poly` still retries up to three times with a quarter of the minimum E and half the time.
Use `1` for msieve's rule.
Source `gnfs/poly/poly_skew.c:904-913, 925-929`; retry `gnfs/poly/poly.c:216-233`.

MSIEVE_ROOTOPT_THREADS, MSIEVE_ROOTOPT_SLOTS

Builds CUDA=.
Values numbers (strtol), clamped to 1–65536.
Default threads: online CPUs divided by the MPI processes on the node (at least one per GPU lane); slots: 256 polynomials per thread.
Effect Threads of the GPU root optimization, divided among the GPUs, and coroutine slots per thread. The job queue holds 4 × slots × lanes.
Source gnfs/poly/stage2/root_sieve_gpu.c:345-351, 2422-2444.

MSIEVE_ROOTOPT_BATCHED

Builds all; two meanings.
Values GPU engine: strcmp "0". CPU root sieve: strcmp "1".
Default on in the GPU engine; off in the CPU code.
Effect In the GPU engine, 0 uses msieve's unbatched root sieve (one call at a time: slow; all kernels except optimize_basic are then unused). In the CPU code (MSIEVE_GPU_ROOTOPT=0 or a CPU build), 1 runs the batched root sieve on the CPU; its results must equal the GPU engine's bit for bit.
Use [test/debug] 1: the bit-identity reference for GPU root optimization. 0 in the engine [slow].
Source root_sieve_gpu.c:2293-2295; gnfs/poly/stage2/root_sieve_batch.c:63-73.

MSIEVE_ROOTOPT_CPU_SHARE

Builds CUDA=.
Values auto, or a number (atof) clamped to $[0, 1]$. A non-number other than auto reads as 0.
Default auto.
Effect The share of each batch's optimize_basic() calls (and Murphy E candidates) done on the CPU thread itself. auto starts at 0 and adapts per thread: +0.02 (up to 0.95) after waiting more than 0.5 ms for the GPU, −0.01 otherwise.
Source root_sieve_gpu.c:2297-2303, 1581-1586.

MSIEVE_GPU_LINESIEVE

Builds CUDA=; batched engine.
Values strcmp "0".
Default on, if the device allows at least 67,584 bytes of opt-in shared memory per block and the PTX has ls_sieve.
Effect The batched line sieve on the GPU; 0: on the CPU.
Source root_sieve_gpu.c:2305-2324.

MSIEVE_GPU_LINECAND

Builds CUDA=; batched engine.
Values strcmp "0".
Default on (chosen for the target hardware).
Effect On: the line candidates' optimize_basic() calls stay on the GPU. 0: they go into the ordinary batches, of which MSIEVE_ROOTOPT_CPU_SHARE runs on the CPU.
Use On the RTX PRO 4500 (FP64 1/64) 0 was faster (a c155 coefficient 104 s against 112–117 s). Whether on is faster on the RTX PRO 6000 or B200 is an untested estimate.
Source root_sieve_gpu.c:2327-2337.

MSIEVE_GPU_MURPHY

Builds CUDA=; batched engine.
Values strcmp "0".
Default on (chosen for the target hardware).
Effect Murphy E optimization of each root heap's candidates on the GPU (one warp per candidate, low-priority stream). 0: on the CPU. E agrees within about 10^{-8}; translations can differ where E is flat.
Use On the RTX PRO 4500 the GPU is already saturated: a c155 coefficient took 107 s with 0 and 170–183 s with it on. With MSIEVE_GPU_LINECAND=0 too, 168 s. Set both to 0 on small or FP64-poor cards. That on is faster on the RTX PRO 6000 or B200 is an untested estimate (the RTX PRO 6000 Blackwell also has FP64 at 1/64).
Source root_sieve_gpu.c:2092-2111.

MSIEVE_GPU_ALPHA

Builds CUDA=; batched engine.
Values strcmp "0".
Default on, if the Murphy prime bound is msieve's PRIME_BOUND.
Effect Root properties (alpha) of every rotation in a root heap on the GPU (CGBN); bit-identical. 0: on the CPU, which also turns off MSIEVE_GPU_BERNSTEIN.
Source root_sieve_gpu.c:2188-2199.

MSIEVE_GPU_BERNSTEIN

Builds CUDA=; batched engine; needs GPU alpha.
Values strcmp "0".
Default on.
Effect The Bernstein size score after alpha on the GPU (within 1 ulp). 0: on the CPU.
Source root_sieve_gpu.c:2163-2182.

MSIEVE_GPU_RSSETUP

Builds CUDA=; batched engine.
Values strcmp "0".
Default on.
Effect Root sieve setup (residues mod the prime powers, roots in the CPU's order) on the GPU; the roots stay there. 0: on the CPU, which also turns off the GPU lattice choice (GPU_LATTICE).
Source root_sieve_gpu.c:2116-2128.

MSIEVE_ROOTOPT_STATS [measurement]

Builds CUDA=.
Values presence.
Effect At the end of root optimization, one line per engine thread on stdout (polynomials, batches, CPU/GPU/idle/setup/teardown time, most lines).
Source root_sieve_gpu.c:2613.

The engine logs which parts run where in one line ("GPU root optimization on ...: ... line sieve on the GPU, lattice choice on the GPU, ..."; root_sieve_gpu.c:2494-2504). Check that line after changing any of these settings.

4.4 Line siever

MSIEVE_GPU_SIEVE

Builds CUDA=.
Values the atoi test, in both readers (the lattice choice and the GPU line siever): 0, no, off turn it off. (Until this fork's last fix the GPU line siever compared with "0" only, so no gave the GPU line siever.)
Default on: -ns runs the GPU lattice siever.
Effect off: -ns runs msieve's CPU line siever (unless the option string contains the word lattice).
Source gnfs/sieve/sieve_line.c:90-97; gnfs/sieve/sieve_line_gpu.c:625, 641.

MSIEVE_SIEVE_THREADS

Builds CUDA=; GPU line and lattice sievers.
Values number (atoi), at least 1.
Default online CPUs divided by the MPI processes on the node.
Effect Host threads that trial-divide and check candidates.
Source sieve_line_gpu.c:725-735; sieve_lattice_gpu.c:1206-1216.

MSIEVE_SIEVE_DUMP [test/debug]

Builds all; CPU line siever and GPU line siever.
Values file name, opened for writing.
Effect Every candidate (b a nlp_r nlp_a), to compare the two line sievers.
Source sieve_line.c:282-285; sieve_line_gpu.c:626.

4.5 Lattice siever

The GPU lattice siever is the default of -ns in GPU builds. Its PTX is checked (LA_ABI); a run that cannot use it falls back to the line siever only when it was the default (section 5).

MSIEVE_LATTICE_LOGI

Values number (atoi), clamped to 9–14.
Default 12: $I = 4096$, $J = I/2 = 2048$.
Effect $\log_2 I$ of the sieve region per lattice; memory per lattice grows with $I \cdot J$.
Source gnfs/sieve/sieve_lattice_gpu.c:1033-1038.

MSIEVE_LATTICE_TINY

Values number (atoi), at most 256 (LA_RESIEVE_MIN).
Default 32.
Effect Primes below it are not sieved; their expected contribution lowers the thresholds. 0: sieve every prime. On the c95, 32 made la_tile 2.3× faster with as many relations.
Source sieve_lattice_gpu.c:1085-1092.

MSIEVE_LATTICE_BATCH

Values number (atoi); raised to at least 8 (LA_MAX_DEG); no upper limit.
Default min(64, 0.4 · free GPU memory/bytes per lattice).
Effect Lattices per GPU batch. A value too large for the memory makes an allocation fail and msieve exit.
Source sieve_lattice_gpu.c:1102-1117.

MSIEVE_LATTICE_FILTER

Values atoi test.
Default on, when the rational polynomial is linear with coefficients below 120 bits, the rational large-prime bounds are below 127 bits and long is 64-bit.
Effect la_filter: the exact 128-bit rational cofactor test on the GPU (removes about two thirds of the candidates). 0: every candidate goes to the host, and la_cofactor is off too.
Use [test/debug] [slow] for comparison.
Source sieve_lattice_gpu.c:812-826.

MSIEVE_LATTICE_COFACTOR

Values atoi test.
Default on, when the filter is on, the algebraic degree is at most 8, its coefficients are below 248 bits and the algebraic large-prime bounds below 127 bits.
Effect la_cofactor (CGBN 256-bit): algebraic norm, the host's large-prime bounds and the Fermat tests on the GPU; only about 2.8% of la_filter's survivors reach the host. 0: those tests on the host.
Use [test/debug] [slow] for comparison.
Source sieve_lattice_gpu.c:838-850.

MSIEVE_LATTICE_CHECK [test/debug]

Values `atoi` test.
Default off.
Effect Before sieving, the sieve sums of one lattice (the first prime $q \geq q_0$ with a root, its first root) are computed on the host at sampled positions, directly from $a \equiv Rb \bmod p$, and compared with the GPU's; the result is logged and sieving continues.
Use After kernel changes. It cannot see what both it and the GPU skip; use MSIEVE_LATTICE_VERIFY as well.
Source `sieve_lattice_gpu.c:1239-1253`; `la_check` at `:890`.

MSIEVE_LATTICE_VERIFY [test/debug]

Values `atoi` test.
Default off.
Effect Every cofactor is checked (gcd with the product of the factor-base primes ≥ 256) for a prime the GPU did not report; the first five are printed on stderr and the total is logged. There must be none. It adds one large gcd per cofactor (not timed here).
Use After kernel changes.
Source `sieve_lattice_gpu.c:1187-1199, 519-531, 1332-1336`.

MSIEVE_BATCH_DUMP [test/debug]

Builds `CUDA=`; lattice siever only.
Values file name, opened for writing.
Effect Writes `a b rational_cofactor algebraic_cofactor` for every relation sent to batch factoring, for `test/sieve/batchreplay.c`, which replays them in shuffled orders (batch factoring must give the same relations in any order).
Source `sieve_lattice_gpu.c:1184-1186, 549-552`.

4.6 Batch factoring

MSIEVE_BATCH_THREADS

Builds non-Windows; all sievers (CPU and GPU).
Values number (`atol`).
Default online CPUs.
Effect The remainder tree splits its halves over threads down to depth $\lfloor \log_2 n \rfloor$, so the thread count is in effect rounded down to a power of two; 0 or 1: one thread. Output stays under a mutex.
Source `common/batch_factor.c:456-470`.

4.7 Filtering and the relation store

MSIEVE_FILTER_THREADS

Builds non-Windows.
Values number (`atoi`), clamped to 1–256.
Default online CPUs.
Effect Threads for: parsing relation blocks in the filtering passes and the cycle reader (matrix, square root); the relation store's first parse; in-memory singleton passes (only with at least 65,536 entries); testing free relations (only with at least 4096 candidates). The output does not depend on it (the logged pass count of singleton removal may).
Source `gnfs/filter/relpar.c:31-51`; `common/filter/singleton.c:472-478`; `gnfs/relstore.c:347-350`; `gnfs/sieve/sieve_util.c:300-304`.

MSIEVE_RELSTORE

Builds all.
Values `atoi` test.
Default on.
Effect On: when this process starts a new savefile, every relation written to it (and the free relations) is also kept in memory as text, parsed once on all CPUs at the first filtering pass, and every later pass, the matrix build and the square root read it. `0`: everything reads the savefile. A run on an existing savefile (`-nc` in a new process, a restart) never uses the store.
Use [test/debug] `0` to verify the store against the file path: after the first pass's count (which differs by the free relations), everything must match.
Source `gnfs/relstore.c:136-140`; created in `gnfs/gnfs.c`.

MSIEVE_RELSTORE_MB

Values number of MiB (`strtoull`).
Default a quarter of physical memory (non-Windows); 4 GiB on Windows.
Effect Memory limit of the store. Past it the store is dropped (logged) and everything reads the savefile. `0` drops it at once.
Source `gnfs/relstore.c:160-174`.

4.8 Linear algebra

MSIEVE_GPU_LA

Builds `CUDA=`.
Values `strcmp "0"`.
Default on.
Effect Block Lanczos on the GPU when the matrix has at least `MSIEVE_GPU_LA_MIN` columns, a device is usable, `lanczos_core_sm*.ptx` loads, and the matrix twice plus nine vectors fit in 90% of the free GPU memory; otherwise the CPU (logged). `0`: always the CPU.
Source `common/lanczos/lanczos_dispatch.c:23-33`; size check `common/lanczos/gpu/lanczos_gpu.c:142-160`.

MSIEVE_GPU_LA_MIN

Builds `CUDA=`.
Values number of columns (`strtoul`).
Default 100000.
Source `lanczos_dispatch.c:24-25`.

MSIEVE_LA_CUDA_MPI

Builds `CUDA=` and `MPI=1`.
Values `strcmp "0"` (it can only turn the mode off).
Default on if the MPI library reports CUDA support at compile time (`MPIX_CUDA_AWARE_SUPPORT` in `mpi-ext.h`) and at run time (`MPIX_Query_cuda_support()`); off otherwise (Fedora's Open-MPI).
Effect On: the MPI grid operations pass device vectors and the grid helpers XOR/copy on the GPU. `0`: through pinned host buffers. Gathers and scatters always use host buffers.
Use `0` to compare, or if a CUDA-aware MPI misbehaves. 600k matrix on a 1×2 grid sharing one card: 36.4 s on, 40.6 s off; on a 200k matrix off was faster (8.4 vs 11.2 s).
Source `common/lanczos/gpu/lanczos_gpu.c:267-285`.

MSIEVE_LA_NCCL

Builds CUDA=, MPI=1, NCCL=1; only with more than one MPI process.
Values strcmp "1".
Default off.
Effect The per-iteration grid operations go through NCCL (ncclSend/ncclRecv rings with a GPU XOR, ncclAllGather). All processes must agree, or all fall back. NCCL refuses two processes on one GPU, so one GPU per process is needed; a GPU-UUID check runs first.
Use Multi-GPU nodes. Untested with NCCL itself (single-GPU development machine).
Source common/lanczos/gpu/lanczos_ring.c:282-301.

MSIEVE_LA_RING_TEST [test/debug]

Builds CUDA= and MPI=1; more than one MPI process.
Values strcmp "1"; takes precedence over MSIEVE_LA_NCCL.
Default off.
Effect The ring logic with MPI_Sendrecv on host buffers instead of NCCL: the only way to test the rings on one GPU.
Source lanczos_ring.c:286, 292-293.

4.9 Square root

MSIEVE_SQRT_THREADS

Builds non-Windows (one dependency at a time otherwise).
Values number (atoi); then at most the online CPUs, and 1–64.
Default 4; 8 when the machine has 32 or more online CPUs.
Effect Dependencies tried at once. Their relations are read in one pass; results are taken in dependency order with the serial stopping rule, so the factors found are the same.
Use With 8 on 32 CPUs a round took about as long as 4, without the occasional second round (c120: 64 s → 34 s in the run that needed one).
Source gnfs/sqrt/sqrt.c:555-577.

MSIEVE_SQRT_GPU

Builds CUDA=; 64-bit GMP limbs.
Values strcmp "0".
Default on.
Effect Every mpz_mul of the square root goes through sqrt_mpz_mul: on the GPU (NTT modulo three primes $c \cdot 2^{40} + 1$) when both operands have at least MSIEVE_SQRT_GPU_MIN limbs, GMP otherwise. 0: GMP only. The GPU used is the process's first selected device.
Use It saved 17% of the square root time at c130 and c140; most time is relation reading and small products.
Source gnfs/sqrt/sqrt_gpu_mul.c:167-177.

MSIEVE_SQRT_GPU_MIN

Builds CUDA=.
Values number of 64-bit limbs (atol); used only if > 0.
Default 2048.
Source sqrt_gpu_mul.c:182-184.

MSIEVE_SQRT_GPU_CHECK [test/debug] [slow]

Builds CUDA=.
Values atoi test.
Default off.
Effect Every GPU product is recomputed with GMP and compared; a difference prints a message and exits. (c120: 38,688 of 38,688 equal.)
Source sqrt_gpu_mul.c:185-186, 536-546.

4.10 GPU selection, several GPUs, MPI

Device selection is by `-g` (section 5) and `gpu_select()` (`common/cuda_xface.c:131-165`). Without `-g` a process uses every visible device; with MPI, local rank r of L processes on a node takes devices $r, r+L, \ldots$; with fewer GPUs than processes they share (rank r takes device $r \bmod$ GPUs) and stage 1 divides that card's memory budget among them. **Only polynomial selection (stage 1 and root optimization) uses several GPUs in one process.** The sievers, the linear algebra and the square root use the first device of the process's set.

`MSIEVE_SIMULATE_GPUS` [test/debug]

Builds `CUDA=`; stage 1 and root optimization.
Values number. Stage 1: `atoi`, used if > 0, capped at 64. Root optimization: `strtol` clamped to 1–65536, then to 64.
Default the number of selected devices.
Effect n logical GPUs (lanes) mapped onto the physical ones as $i \bmod$ devices; the lanes of one card share its memory budget. Results must not depend on it.
Use Testing the multi-GPU paths on one card; it cannot show a speedup. Note: `0` is ignored by stage 1 but gives one lane in root optimization.
Source `stage1_sieve_gpu.c:1839, 1861-1865`; `root_sieve_gpu.c:2399-2400`.

`CUDA_VISIBLE_DEVICES` (CUDA driver)

Effect Not read by msieve: the driver hides devices before `cuDeviceGetCount`, so it restricts every part, and `-g N` counts within the visible set. It also decides whether a CUDA device is "there" for the NFS default (section 4.11).

`OMPI_COMM_WORLD_LOCAL_RANK and similar`

Effect Not read. The node-local rank and size come from `MPI_Comm_split_type(MPI_COMM_WORLD, MPI_COMM_TYPE_SHARED, ...)` (`common/driver.c:252-266`), which works with any MPI.

`OMPI_CC` (build time)

Effect Read by OpenMPI's `mpicc` wrapper, not by msieve. With HPC-X set it to `gcc`.

MPI polynomial selection: process r of R takes every R-th leading coefficient accepted by the a_d sieve; `-nps`/`-npr` split their input file by line. Rank 0 writes the usual files; rank $r > 0$ writes `<name>.m|ms|p.mpiNN` and `<log>.mpiNN`, and rank 0 merges them over MPI at the end (no shared file system needed for these). Launch with `mpirun --bind-to none`.

4.11 The NFS by default

`MSIEVE_DEFAULT_NFS`

Builds `CUDA=`.
Values `atoi` test (so `no`, `off` or empty also mean off).
Default on.
Effect When no `-n*` option is given, inputs of at least `MSIEVE_NFS_MIN_DIGITS` digits and more than `MIN_NFS_BITS` bits are factored with the whole NFS, if `cuInit` succeeds and at least one CUDA device is visible (logged: "GPU build: the number field sieve by default"). `0`: msieve's MPQS as before.
Use c80–c84 by NFS took 73–88 s against 132–382 s by MPQS; c95 153 s against 2,800 s.
Source `common/driver.c:86-114, 187-204`.

`MSIEVE_NFS_MIN_DIGITS`

Builds `CUDA=`.
Values number of decimal digits (`atoi`).
Default 80 (`NFS_DEFAULT_DIGITS`).
Effect Smallest input for the NFS default. Values below about 80 change nothing, because inputs of at most `MIN_NFS_BITS` (264) bits never use the NFS.
Source `common/driver.c:95-99`.

4.12 Testing and debugging: summary

Setting	Purpose
`MSIEVE_CPU_LIFT`, `MSIEVE_OLD_CRT`	old stage 1 code paths, to compare
`MSIEVE_SIZEOPT_LOG`	size-optimization survivors and root optimization results
`MSIEVE_ROOTOPT_BATCHED=1`	batched root sieve on the CPU: bit-identity reference
`MSIEVE_ROOTOPT_STATS`	per-thread engine statistics
`MSIEVE_SIMULATE_GPUS`	multi-GPU paths on one card
`MSIEVE_SIEVE_DUMP`	line siever candidates, CPU vs GPU
`MSIEVE_LATTICE_CHECK`, `MSIEVE_LATTICE_VERIFY`	lattice siever sums and missed primes
`MSIEVE_BATCH_DUMP`	batch-factoring inputs for `test/sieve/batchreplay.c`
`MSIEVE_RELSTORE=0`	store vs savefile path
`MSIEVE_LA_RING_TEST`	NCCL ring logic over MPI
`MSIEVE_SQRT_GPU_CHECK`	GPU products vs GMP (slow)
`-DMIN_NFS_BITS`	NFS below 80 digits

Test tools in the tree: `test/lanczos/` (`matgen`, `depcheck`, `optest`, `run.sh`; synthetic matrices and `-nc2 "skip_matbuild=1"`), `test/sieve/batchreplay.c`, and `test/sieve/relcheck.py <fb> <savefile>` (validates every relation).

5 Command-line options

Only the options whose meaning the fork changed or extended are listed; see `Readme` and `Readme.nfs` for the rest.

-g <num>	`CUDA=` builds. Use GPU `num` only. The default is now every visible GPU (`MSIEVE_ALL_GPUS` = `0xffffffff`, `demo.c:333`; library callers pass it as `which_gpu`, `include/msieve.h:190`). With MPI, `-g` puts every process of the node on that GPU.
-t <num>	Stage 1 runs between one and four threads per GPU: max(lanes, min(4·lanes, t)) for `-t` t; without `-t`, one per GPU (`stage1_sieve_gpu.c:1867-1869`). Other uses are upstream's.
-n	All NFS steps. In `CUDA=` builds the NFS also runs without any `-n*` option for inputs of 80 digits or more (section 4.11).
-np, -np1, -nps, -npr	As upstream (stage 1 + size + root; stage 1; size; root). In GPU builds size optimization runs on the GPU inside `-np1`, so `-np1 -nps` together is the fast form; `-nps`/`-npr` alone on files use the stage 2 threads. All `-n*` options share one option string: put every `name=value` in one quoted argument. `gpu_mem_mb=` (upstream) is divided among MPI processes sharing a card and among the card's stage 1 threads.
-ns ["q0,q1"]	GPU builds: the GPU lattice siever, special-q from `q0` to `q1` (raised to above the algebraic factor base, at most $2^{32} - 1$). Without a range it starts at the end of the factor base or at the resume point in `<savefile>.lattice`.
-ns "line [b0,b1]"	The GPU line siever over lines b (`MSIEVE_GPU_SIEVE=0`: the CPU line siever).
-ns "lattice q0,q1"	The lattice siever explicitly; also in a build where it is not the default. If it fails there is no fallback ("lattice sieving failed"). When the lattice siever runs by default and cannot start, the line siever runs instead, and a given range is then taken as a range of b (`sieve_line.c:236-264`). In a CPU build `lattice` is an error.
-nc, -nc1, -nc2, -nc3, -ncr	As upstream. Note (upstream behaviour, `demo.c:472-478`): every `-n*` option takes the next argument as its option string if that does not start with `-`. `./msieve -nc1 <N>` therefore swallows N, and msieve reads the number from `worktodo.ini`. Give an option string (e.g. `-nc1 ""`) before N, or use `worktodo.ini`.
MPI	No new options. Polynomial selection and the GPU Lanczos use all processes; the Lanczos grid is `-nc2 "mpi_nrows=R mpi_ncols=C"` as upstream (`Readme.nfs`).
-v	Stage 1 hits are echoed to stdout only with `-v`.

Always pass `-nf <name>.fb` when testing polynomial selection: a run that finds an existing `msieve.fb` skips polynomial selection.

6 README and code disagreements

Where `README.md` and the code differ, the code is right; this document follows the code.

- Fixed together with this document: the README's square root section gave 4 as the default of `MSIEVE_SQRT_THREADS` (8 with 32 CPUs or more).
- Fixed together with this document: a `CUDA=12` build wrote PTX only for sm_60–75, so on compute 8.0 and newer stage 1 stopped; it now also builds sm_80–120.
- The README's build section names only `stage1_core` and `sizeopt_core` PTX files as loaded from the current directory. Six families are (table 3); only `lasieve_core` and `sqrtmul_core` are checked against the binary.

- Fixed together with this document: `MSIEVE_GPU_SIEVE=no` selected the GPU line siever (its reader compared with `"0"`); both readers now use `atoi`.
- The README gives no range for `MSIEVE_LATTICE_LOGI` (clamped to 9–14) or `MSIEVE_LATTICE_TINY` (at most 256), and does not say that `MSIEVE_LATTICE_FILTER=0` also turns off `la_cofactor`.

7 Alphabetical index

Name	Component	Default	Effect
`CGBN_INCLUDE` (make)	build	`/usr/local/include`	CGBN headers
`CUDA` (make)	build	CPU build	`12`/`13`: GPU build, PTX set
`CUDA_VISIBLE_DEVICES`	driver	all	devices msieve can see
`ECM` (make)	build	off	GMP-ECM
`GPU_LATTICE` (make)	build	1	root sieve lattice choice on GPU
`LA_TILE_POS` (-D)	lattice	8192	`la_tile` positions
`MACHINE_FLAGS` (make)	build	empty	extra C flags, `-D` macros
`MIN_NFS_BITS` (-D)	driver	264	smallest NFS input
`MPI` (make)	build	off	MPI build
`NCCL`, `NCCL_HOME` (make)	build	off	NCCL for GPU Lanczos
`NO_ZLIB`, `BOINC`, `WIN`, `WIN64`	build	off	upstream
`NVCC` (make)	build	`which nvcc`	CUDA toolkit
`Q_ROOTS_MIN` (-D)	stage 1	degree	fewest special-q roots
`VBITS` (make)	LA	64	Lanczos vector width
`MSIEVE_BATCH_DUMP`	lattice	unset	dump batch inputs (test)
`MSIEVE_BATCH_THREADS`	batch	CPUs	remainder tree threads
`MSIEVE_CPU_LIFT`	stage 1	unset	CPU lifting (test)
`MSIEVE_DEFAULT_NFS`	driver	on	NFS without `-n`
`MSIEVE_FILTER_THREADS`	filtering	CPUs	parse/filter threads
`MSIEVE_GPU_ALPHA`	root opt	on	alpha on GPU
`MSIEVE_GPU_BERNSTEIN`	root opt	on	size score on GPU
`MSIEVE_GPU_LA`	LA	on	Lanczos on GPU
`MSIEVE_GPU_LA_MIN`	LA	100000	smallest GPU matrix
`MSIEVE_GPU_LINECAND`	root opt	on	line candidates on GPU
`MSIEVE_GPU_LINESIEVE`	root opt	on	line sieve on GPU
`MSIEVE_GPU_MURPHY`	root opt	on	Murphy E on GPU
`MSIEVE_GPU_ROOTOPT`	root opt	on	GPU root optimization
`MSIEVE_GPU_RSSETUP`	root opt	on	root sieve setup on GPU
`MSIEVE_GPU_SIEVE`	sieving	on	lattice / GPU line siever
`MSIEVE_GPU_SIZEOPT`	size opt	on	size optimization on GPU
`MSIEVE_LA_CUDA_MPI`	LA, MPI	auto	device buffers to MPI
`MSIEVE_LA_NCCL`	LA, MPI	off	NCCL rings
`MSIEVE_LA_RING_TEST`	LA, MPI	off	rings over MPI (test)
`MSIEVE_LATTICE_BATCH`	lattice	≤ 64	lattices per batch
`MSIEVE_LATTICE_CHECK`	lattice	off	sieve sum check (test)
`MSIEVE_LATTICE_COFACTOR`	lattice	on	`la_cofactor`
`MSIEVE_LATTICE_FILTER`	lattice	on	`la_filter`
`MSIEVE_LATTICE_LOGI`	lattice	12	$\log_2 I$
`MSIEVE_LATTICE_TINY`	lattice	32	unsieved small primes
`MSIEVE_LATTICE_VERIFY`	lattice	off	missed-prime check (test)
`MSIEVE_NFS_MIN_DIGITS`	driver	80	NFS default from
`MSIEVE_OLD_CRT`	stage 1	unset	old CRT (test)
`MSIEVE_POLY_KEEP_E`	root opt	0.25	keep down to min E $\times$
`MSIEVE_POLY_TIME_SCALE`	poly	1/128	time limit factor
`MSIEVE_Q_THREADS`	stage 1	4	special-q producers

Name	Component	Default	Effect
MSIEVE_RELSTORE	filtering	on	in-memory relations
MSIEVE_RELSTORE_MB	filtering	RAM/4	store limit
MSIEVE_ROOTOPT_BATCHED	root opt	GPU on	batched root sieve
MSIEVE_ROOTOPT_CPU_SHARE	root opt	auto	CPU share of calls
MSIEVE_ROOTOPT_SLOTS	root opt	256	polys per thread
MSIEVE_ROOTOPT_STATS	root opt	unset	thread statistics
MSIEVE_ROOTOPT_THREADS	root opt	CPUs	engine threads
MSIEVE_ROOTOPT_TOPK	root opt	48 / 0	root-optimize K best
MSIEVE_SIEVE_DUMP	line siever	unset	candidate dump (test)
MSIEVE_SIEVE_THREADS	sieving	CPUs	host sieve threads
MSIEVE_SIMULATE_GPUS	multi-GPU	devices	logical GPUs (test)
MSIEVE_SIZEOPT_LOG	size opt	unset	survivor log
MSIEVE_SQRT_GPU	sqrt	on	GPU products
MSIEVE_SQRT_GPU_CHECK	sqrt	off	compare with GMP (slow)
MSIEVE_SQRT_GPU_MIN	sqrt	2048	limbs for GPU product
MSIEVE_SQRT_THREADS	sqrt	4 or 8	dependencies at once
MSIEVE_STAGE2_THREADS	poly	CPUs	size/root workers